

VULNERABILITY ASSESSMENT REPORT

Target Application: Altoro Mutual Online Banking

Target URL: <http://demo.testfire.net>

IP Address: 65.61.137.117

Assessment Type: External Black-Box Assessment

Assessment Date: 19-25 May 2026

Prepared By: Sandile Theron Lubisi

Institution: University of the Western Cape (UWC)

Internship: Future Interns — Cyber Security Track

CONFIDENTIALITY STATEMENT

This document is the exclusive property of Sandile Theron Lubisi and Future Interns. It contains proprietary and confidential information related to the security assessment of demo.testfire.net. This document may only be shared with authorized parties under non-disclosure agreements.

DISCLAIMER

A vulnerability assessment is a snapshot in time. The findings and recommendations in this report reflect information gathered only during the assessment period and do not account for changes made outside of that timeframe.

This assessment was conducted on a publicly available demo application provided by IBM/HCL Technologies specifically for security testing purposes. **No production systems were accessed, exploited, or harmed during this assessment.**

It is recommended that similar assessments be conducted on an annual basis to ensure the continued effectiveness of security controls as the threat landscape evolves.

CONTACT INFORMATION

Assessor:	Sandile Theron Lubisi
Institution:	University of the Western Cape (UWC)
Programme:	BSc Computer Science — 3rd Year
Internship:	Future Interns Cyber Security Track

Contact:

072 873 2592

ASSESSMENT OVERVIEW

From May 19 to May 25, 2026, a vulnerability assessment was conducted on the Altoro Mutual demo web application. All testing was performed based on the following industry frameworks:

- OWASP Testing Guide (v4)
- NIST SP 800-115 Technical Guide to Information Security Testing
- OWASP Top 10 (2021)

Phases of Assessment

1. Planning	Define scope, objectives, and rules of engagement.
2. Discovery	Scan and enumerate open ports, services, and potential vulnerabilities.
3. Analysis	Manually verify findings using browser testing and source inspection.
4. Reporting	Document all findings with evidence, business impact, and remediation steps.

Tools Utilized

- **Nmap 7.98:** Port scanning, service enumeration, and vulnerability scripting (NSE).
- **OWASP ZAP 2.17.0:** Automated passive web vulnerability scanning and sequence monitoring.
- **Chrome DevTools:** Manual HTTP header analysis, cookie inspection, and DOM validation.

FINDING SEVERITY RATINGS

SEVERITY	DEFINITION
CRITICAL	Flaw allows for immediate, complete system compromise or data theft. Patch immediately.
HIGH	Vulnerability is easily exploitable and could result in significant data loss or disruption.
MEDIUM	Vulnerability exists and may be exploited under certain conditions or chained with other flaws.
LOW	Unlikely to be exploited alone but reduces the overall security posture. Address during routine maintenance.
INFO	No immediate vulnerability. Noteworthy observations that provide context or best-practice deviations.

SCOPE OF ASSESSMENT

Assessment Type:	External Black-Box Assessment
Target:	http://demo.testfire.net
IP Address:	65.61.137.117
Server:	Apache Tomcat / Coyote JSP 1.1
Open Ports:	80 (HTTP), 443 (HTTPS), 8080 (HTTP-Proxy)

EXECUTIVE SUMMARY

During 19–24 May 2026, a comprehensive vulnerability assessment was conducted on the Altoro Mutual demo banking application at demo.testfire.net.

The assessment identified **14 security vulnerabilities**. A Critical SQL Injection flaw was manually confirmed on the login page, meaning an attacker could bypass authentication entirely without a valid password to access user accounts. Additionally, the server was found vulnerable to a Denial-of-Service attack and uses outdated encryption that falls below modern cryptographic standards.

The application also suffers from a systemic lack of client-side protections, completely missing essential HTTP security headers (CSP, X-Frame-Options) and failing to secure user session cookies. It is strongly recommended that the Critical and High severity findings be addressed immediately to prevent a data breach, followed by server-wide header configurations to resolve the Medium severity items.

Vulnerability Summary

TOTAL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
14	1	2	6	4	1

ATTACK SUMMARY TABLE

NO.	FINDING	RECOMMENDATION	SEVERITY
1	SQL Injection on Login Form	Use parameterized queries; validate inputs	CRITICAL
2	Slowloris Denial-of-Service	Configure connection limits; deploy WAF	HIGH
3	Weak TLS Encryption (1024-bit)	Upgrade to 2048-bit minimum; use ECDHE	HIGH
4	No Anti-CSRF Tokens on Forms	Add unique CSRF tokens to all forms	MEDIUM
5	Content Security Policy (CSP) Not Set	Configure CSP header on server responses	MEDIUM
6	Missing Anti-Clickjacking Header	Add X-Frame-Options: DENY to all pages	MEDIUM
7	SQL Source Code Visible in Page	Audit JSP templates; fix rendering errors	MEDIUM
8	Subresource Integrity (SRI) Missing	Add integrity tags to external scripts	MEDIUM
9	Insecure Session Cookie Attributes	Set Secure, HttpOnly, and SameSite flags	MEDIUM
10	Cross-Domain JavaScript Inclusion	Host scripts locally or verify with SRI	LOW
11	Server Version Disclosed in Headers	Suppress Server header in Tomcat config	LOW

NO.	FINDING	RECOMMENDATION	SEVERITY
12	X-Content-Type-Options Missing	Add X-Content-Type-Options: nosniff	LOW
13	HTML Developer Comments in Source	Remove all comments before deploying	LOW
14	Outdated DOCTYPE / Quirks Mode	Update HTML to modern HTML5 standard	INFO

INDIVIDUAL FINDINGS

DESCRIPTION

The login page at `demo.testfire.net` does not properly sanitize what users type into the fields. An attacker can insert special database commands into these fields to trick the system into granting access without knowing a real password.

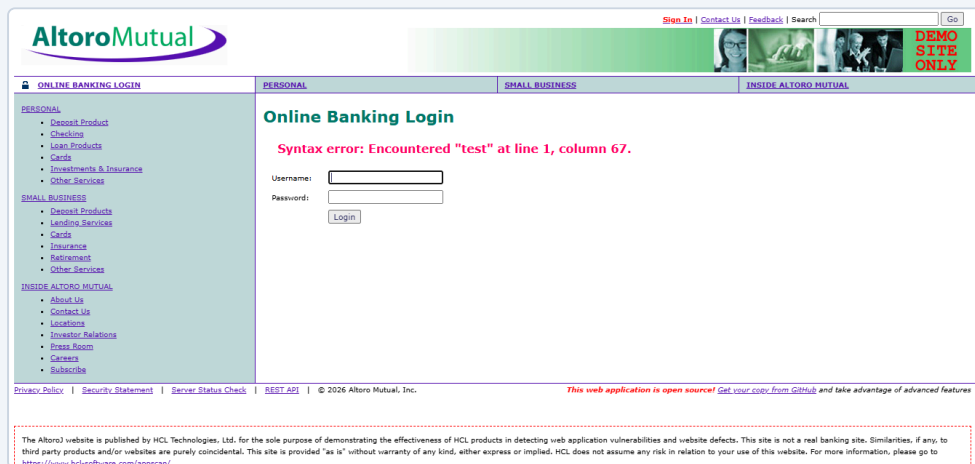
BUSINESS IMPACT

An attacker could bypass the login page entirely. All customer bank accounts, balances, and personal information could be accessed, modified, or deleted, resulting in severe POPIA regulatory penalties.

PROOF OF CONCEPT

Username input provided: `admin'`

Result: Server returned database error "Syntax error:
Encountered 'test' at line 1"



RECOMMENDATION

- Use parameterized queries (prepared statements) so user input is never treated as executable database code.
- Show users generic error messages only — never display raw database error details.

DESCRIPTION

The web server is vulnerable to "Slowloris." An attacker opens hundreds of slow, incomplete connections to the server and holds them open, eventually exhausting the server's connection pool so real users cannot access the website.

BUSINESS IMPACT

An attacker could take the banking website completely offline using a single laptop. Customers would be unable to access their accounts, causing severe reputational and financial damage.

PROOF OF CONCEPT

```
Nmap vulnerability script detected:  
State: LIKELY VULNERABLE  
IDs: CVE:CVE-2007-6750  
Affected Ports: 443, 8080
```

RECOMMENDATION

- Set a maximum number of simultaneous connections allowed per IP address.
- Configure aggressive timeouts for incomplete HTTP requests.
- Deploy a Web Application Firewall (WAF) or reverse proxy (e.g. Cloudflare).

3: Weak Diffie-Hellman Key Exchange

HIGH

DESCRIPTION

The server uses an outdated cryptographic key exchange method with a key size of only 1024 bits. Security experts consider this length too weak to protect data against modern hardware.

BUSINESS IMPACT

A well-resourced attacker could potentially passively intercept and crack this encryption, exposing sensitive data being sent between the bank and its customers (e.g., passwords, transaction details).

PROOF OF CONCEPT

Cipher Suite: TLS_DHE_RSA_WITH_AES_256_CBC_SHA
Modulus Length: 1024 (Weak)
Reference: <https://weakdh.org>

RECOMMENDATION

- Upgrade DH key exchange to a minimum of 2048 bits (preferably 4096 bits).
- Switch to ECDHE (Elliptic Curve Diffie-Hellman) which is stronger and faster.

DESCRIPTION

Cross-Site Request Forgery (CSRF) tricks a logged-in user into performing an action on a website without knowing it. Anti-CSRF tokens are secret codes hidden in forms to verify the action was intentional. This site lacks these tokens.

BUSINESS IMPACT

A customer could be tricked into clicking a malicious link that secretly submits a bank transfer on their behalf without their knowledge or consent.

PROOF OF CONCEPT

OWASP ZAP Sequence confirmed forms lack CSRF protection.
Affected: /login.jsp, /feedback.jsp, /subscribe.jsp

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Absence of Anti-CSRF Tokens	Medium	3
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Missing Anti-clickjacking Header	Medium	Systemic
Source Code Disclosure - SQL	Medium	1
Sub Resource Integrity Attribute Missing	Medium	1
Cookie without SameSite Attribute	Low	4
Cross-Domain JavaScript Source File Inclusion	Low	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	3
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	3
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	3
Dangerous JS Functions	Low	1
Permissions Policy Header Not Set	Low	Systemic
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Information Disclosure - Suspicious Comments	Informational	6
Modern Web Application	Informational	4
Session Management Response Identified	Informational	4
Storable and Cacheable Content	Informational	Systemic

RECOMMENDATION

- Add a unique, random token as a hidden field in every form on the website.
- Verify this token on the server before processing any form submission.

5: Content Security Policy Not Set

MEDIUM

DESCRIPTION

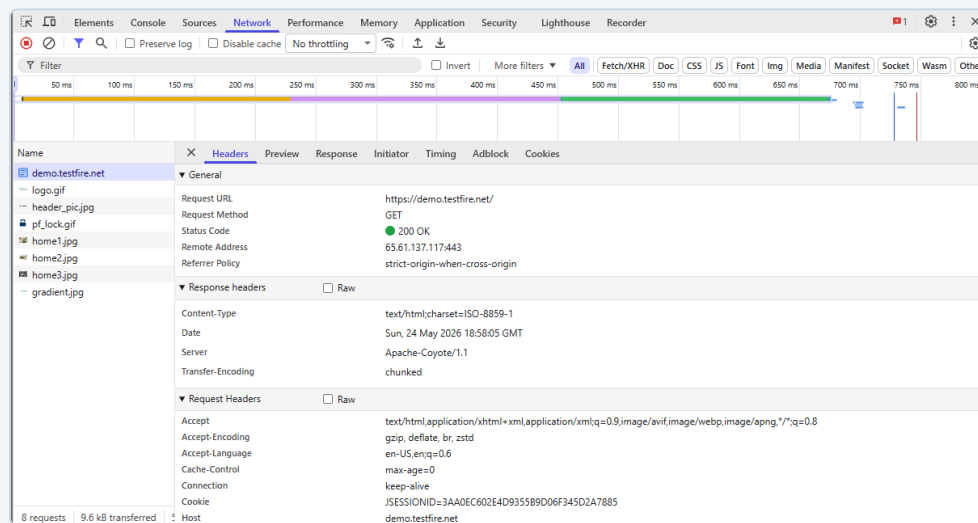
A Content Security Policy (CSP) tells the browser which sources of scripts and content are trusted. Without it, the browser will execute content from any source, including malicious scripts planted by attackers.

BUSINESS IMPACT

The site is significantly more vulnerable to Cross-Site Scripting (XSS) attacks. Attackers can inject malicious JavaScript to steal session tokens or redirect users to phishing pages.

PROOF OF CONCEPT

HTTP Response Headers confirmed systemic absence of Content-Security-Policy.



RECOMMENDATION

- Add the Content-Security-Policy header to all server responses.
- Start with a strict base policy: Content-Security-Policy: default-src 'self'

DESCRIPTION

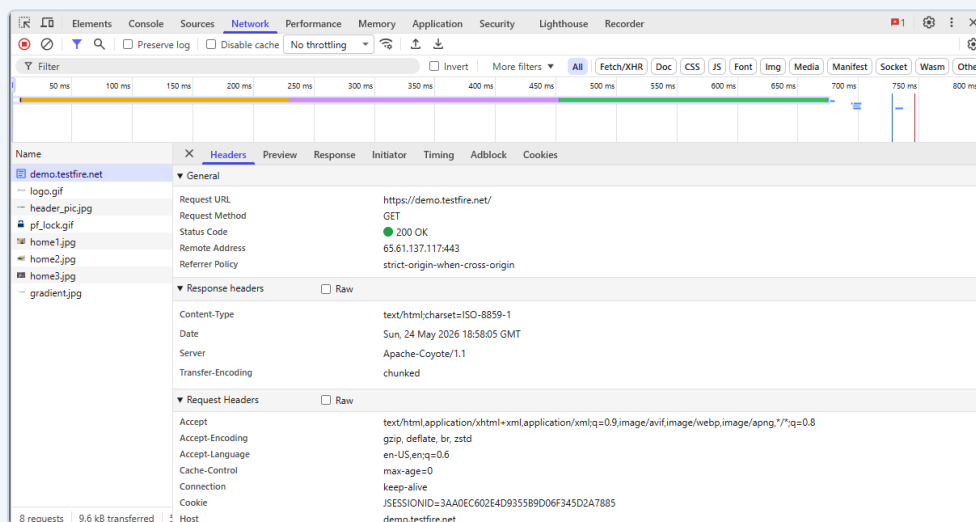
Clickjacking occurs when a criminal website places the banking site invisibly on top of their own page using an iframe. The X-Frame-Options header prevents this embedding, but it is currently missing.

BUSINESS IMPACT

Victims think they are clicking a button on the attacker's page but are actually clicking buttons on the bank's site, potentially authorizing transfers without realizing it.

PROOF OF CONCEPT

HTTP Response Headers confirmed systemic absence of X-Frame-Options.



RECOMMENDATION

- Add the following header to all server responses: X-Frame-Options: DENY.

7: Source Code Disclosure – SQL Fragment

MEDIUM

DESCRIPTION

Parts of the server-side database code were found visible inside a web page response. Database queries should execute privately on the server and never be rendered as text to the user.

BUSINESS IMPACT

When internal code is visible, attackers gain valuable clues about how the database is structured, helping them craft highly targeted SQL Injection attacks.

PROOF OF CONCEPT

Affected URL: `/index.jsp?content=inside_trainee.htm`
ZAP detected SQL fragment in response body: `select "Management Trainee Program" from our`

RECOMMENDATION

- Audit all JSP pages that load content dynamically via URL parameters.
- Implement proper error handling to catch rendering failures before they reach the user.

8: Subresource Integrity (SRI) Missing

MEDIUM

DESCRIPTION

The application loads JavaScript files from external, third-party sources without utilizing Subresource Integrity (SRI) checks. SRI allows the browser to verify that the fetched script hasn't been tampered with.

BUSINESS IMPACT

If the external server hosting the script is compromised, attackers could replace the file with malicious code. The banking app would automatically execute it, compromising users.

RECOMMENDATION

- Add the integrity and crossorigin attributes to all external `<script>` tags.

9: Insecure Session Cookie Attributes

MEDIUM

DESCRIPTION

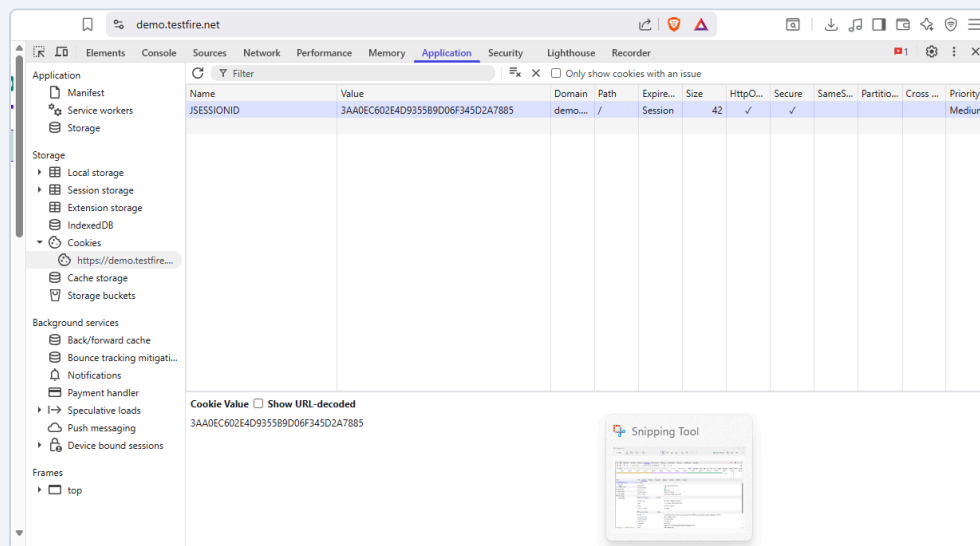
Session cookies are digital IDs proving a user is logged in. DevTools inspection confirmed that the JSESSIONID cookie is missing all three critical security flags: SameSite, HttpOnly, and Secure.

BUSINESS IMPACT

Because the cookie lacks these protections, it can be easily stolen by client-side scripts (XSS), transmitted over unencrypted networks, and used by attackers to execute cross-site forgery attacks.

PROOF OF CONCEPT

Chrome DevTools Application Tab shows JSESSIONID flags are empty.



RECOMMENDATION

- Configure the server's Set-Cookie header to append HttpOnly and Secure flags.
- Set SameSite=Strict to prevent the cookie from being sent with cross-site requests.

DESCRIPTION

The site relies on executing JavaScript files hosted on third-party domains that the bank does not directly control.

BUSINESS IMPACT

This introduces a supply-chain risk. If the third-party domain expires or is hijacked, the new owner gains immediate control over code running on the bank's site.

RECOMMENDATION

- Download necessary third-party scripts and host them locally on the bank's infrastructure.

11: Server Version Disclosed in Header

LOW

DESCRIPTION

The server tells every visitor exactly what software and version it is running. While seemingly harmless, it provides attackers with a reconnaissance shortcut.

BUSINESS IMPACT

Attackers can immediately search exploit databases (CVEs) for known weaknesses specific to this exact software version without having to guess the backend infrastructure.

PROOF OF CONCEPT

```
Nmap and HTTP Headers confirmed:  
Server: Apache Tomcat/Coyote JSP 1.1
```

RECOMMENDATION

- Configure Apache Tomcat to suppress the Server header in `server.xml`.

12: X-Content-Type-Options Header Missing

LOW

DESCRIPTION

The server fails to send the X-Content-Type-Options header. Without this, older browsers may attempt to "sniff" or guess the file type instead of following the server's instructions.

BUSINESS IMPACT

An attacker could upload a malicious HTML script disguised as an image file. The browser might sniff the content, realize it is HTML, and execute the virus.

RECOMMENDATION

- Append X-Content-Type-Options: nosniff to all HTTP responses.

13: HTML Developer Comments in Source

LOW

DESCRIPTION

HTML comments left by developers are invisible on the website but fully visible in the page's raw source code. They are frequently forgotten during production deployment.

BUSINESS IMPACT

While this specific comment is low risk, leaving comments in live code can accidentally reveal sensitive internal system notes, disabled features, or file paths to attackers.

PROOF OF CONCEPT

Source code revealed: `<!-- BEGIN HEADER -->`

```
<!-- BEGIN HEADER -->
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" >
```

RECOMMENDATION

- Implement a build process that automatically strips HTML comments before deployment.

DESCRIPTION

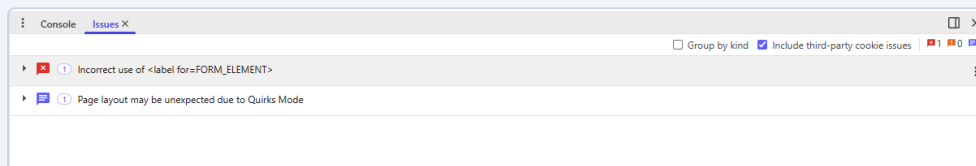
The application does not specify a modern HTML5 DOCTYPE declaration at the top of its pages, forcing the browser to render the site in an outdated "Quirks Mode."

BUSINESS IMPACT

Quirks mode can cause unpredictable rendering issues and may inadvertently disable modern browser security mechanisms, making the site slightly more fragile.

PROOF OF CONCEPT

Chrome DevTools Console flagged Quirks Mode layout issues.



RECOMMENDATION

- Ensure the very first line of every HTML/JSP response is the standard declaration: `<!DOCTYPE html>`.

CONCLUSION

The vulnerability assessment of `demo.testfire.net` identified 14 security findings across High, Medium, Low, and Informational severity levels.

The most critical concern is a manually confirmed **SQL Injection** vulnerability on the login form. An attacker could use this to bypass the authentication page entirely without a valid password, gaining complete access to customer accounts. This issue presents an immediate, critical risk and must be prioritized for patching.

The server is also vulnerable to a Slowloris Denial-of-Service attack and utilizes outdated 1024-bit encryption on its HTTPS connection, both of which fall below modern enterprise standards.

At the application layer, the complete absence of Content Security Policy, X-Frame-Options headers, and Anti-CSRF tokens represents a systemic security gap. Fortunately, these can largely be resolved through sweeping server configuration changes applied once across the entire site.

REMEDIATION PRIORITY PLAN

IMMEDIATE (FIX WITHIN 24–48 HOURS)

- Patch SQL Injection on the login form via parameterized queries.
- Configure Slowloris DoS protection (connection limits/WAF).
- Upgrade TLS DH encryption to 2048-bit minimum.

SHORT-TERM (FIX WITHIN 2 WEEKS)

- Implement Content Security Policy (CSP) header.
- Add X-Frame-Options: DENY header.
- Add Anti-CSRF tokens to all web forms.
- Set Secure, HttpOnly, and SameSite attributes on session cookies.

ROUTINE MAINTENANCE (SCHEDULED UPDATES)

- Suppress Server version header.
- Remove HTML developer comments from production files.
- Update HTML DOCTYPE to HTML5 standards.

Assessed by: Sandile Theron Lubisi May 2026 University of the Western Cape